UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

&str

```
fn main() {
   let s1 = "Departamento\n\"Ciência\" da Computação";
   let s2 = "Departamento \
             Ciência da Computação";
   let s3 = "C:\\Temp";
   let s4 = r"C:\Temp";

   println!("{s1}");
   println!("{s2}");
   println!("{s3}");
   println!("{s4}");
}
```

```
Departamento
"Ciência" da Computação
Departamento Ciência da Computação
C:\Temp
C:\Temp
```

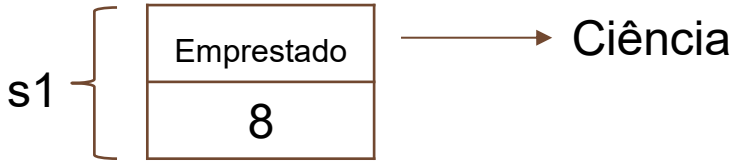
UDESC

&str

s1 (&str) é um *fat pointer* que armazena o endereço da área de memória estática em que foi armazenado o literal "Ciência" e o número de bytes usados para armazenar o literal. O endereço da área de memória estática e "emprestado" para variável s1.

```
fn main() {
    let s1 = "Ciência";
    for c in s1.chars() {
        print!("{c} ");
    }
    for c in s1.bytes() {
        print!("{c} ");
    }
    println!("{s1}");

}
```

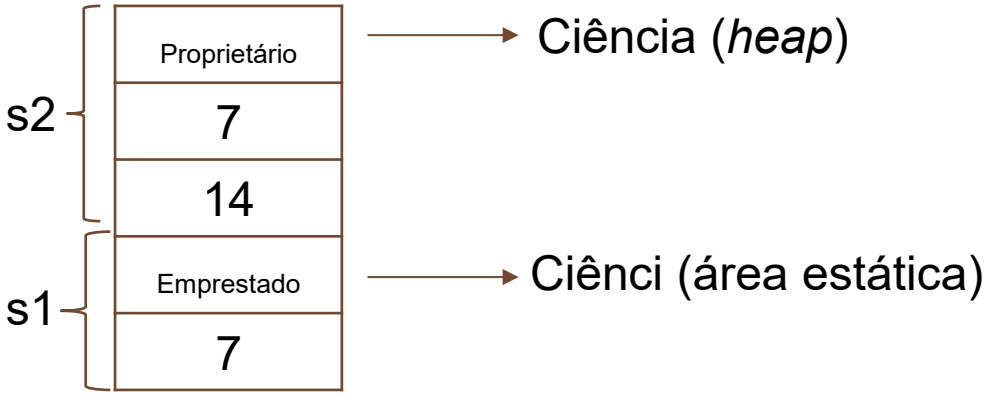



```
C i ê n c i a 67 105 195 170 110 99 105 97 Ciência
```

String

s2 (String) é uma estrutura de dado (Vec<u8>) que armazena o endereço da área de memória onde os bytes da string são armezenados (no *heap*), o número de bytes da *string* e o tamanho da área reservada.

```
fn main() {
   let mut s1 = “Ciênci”;
   let mut s2 = s1.to_string();
   // s1.push('a'); Erro
   s2.push('a');
   println!("{s1} – {s2}");
}
```



UDESC

Ownership

Podemos pensar que ter a "propriedade" de uma área de memória é ter o controle de quando ela será liberada. Na atribuição essa propriedade é transferida, com exceção de dados que são integramente copiados (*e.g.* i32, u64, f64).

```
fn main() {
    let x1 = 54;
    let x2 = x1;
    println!("{x1} - {x2}");
    let s1 = "teste".to_string();
    let s2 = s1;
    println!("{s1} - {s2}");
}

// println!("{s1} - {s2}");
   |                ^^ value borrowed here after move
```

UDESC

Ownership

Regras de propriedade:

- Cada área de memória possui apenas um proprietário;
- Quando o dono sai do escopo a área e liberada.

```
fn foo (s:String) {
    println!("{s}");
}

fn main() {
   let s1 = "teste".to_string();
   foo(s1);
   println!("{s1}"); // Erro
}
```



Ownership

```
fn foo (s:String) -> String {
    println!("{s}");
    s
}

fn main() {
   let mut s1 = "teste".to_string();
   s1 = foo(s1);
   println!("{s1}");
}
```

UDESC

Borrowing

Uma referência armazena o endereço de uma área de memória, a atribuição do endereço de uma variável a outra não transfere a propriedade. Quando isso acontece dizemos que houve um "empréstimo".

```
fn foo (s:&String) {
    println!("{s}");
}

fn main() {
   let s1 = "teste".to_string();
   foo(&s1);
   println!("{s1}");
}
```

UDESC

Borrowing

Uma referência imutável pode ser emprestada várias vezes. Se alguma referência mutável recebe o endereço de uma área de memória, apenas essa referência pode acessar essa área. Quando a variável que recebeu a referência não está mais no escopo o proprietário volta a ter seu acesso.

```
fn main() {
   let s1 = "teste".to_string();
   let s2 = &s1;
   let s3 = &s1;
   println!("{s1} - {s2} - {s3}");
}
```

```
fn main() {
   let mut s1 = "teste".to_string();
   let s2 = &mut s1;
   s2.push('*');
   println!("{s1} - {s2}"); // Erro
}
```

UDESC

Borrowing

Apenas um pode atualizar ou vários podem ler.

```
fn main() {
    let mut s1 = "teste".to_string();
    let s2 = &mut s1;
    s2.push('*');
    println!("{s1} - {s2}"); // Erro
}
```

```
fn foo (s:&mut String)  {
     s.push('*');
}

fn main() {
    let mut s1 = "teste".to_string();
    foo(&mut s1);
    println!("{s1}");
}
```

UDESC

Borrowing

```
fn main() {
   let mut s1 = "Ciência da Computação".to_string();
   s1.push('*’); // Empréstimo mutável
   let s2 = &s1[12..24]; // Empréstimo imutável
   let s3 = &s1[12..25]; // Emprestimo imutável
   println!("{s1} - {s2} - {s3}"); // Emprestimo imutável
}
```

```
Ciência da Computação* - Computação - Computação*
```

UDESC

Borrowing

```
fn main() {
   let mut s1 = "Ciência da Computação".to_string();
   s1.push('*’); // Empréstimo mutável
   let s2 = &s1[12..24]; // Empréstimo imutável
   s1.push('*’); // Empréstimo mutável
   println!("{s1} - {s2}"); // Empréstimo mutável
}
```

```
 let s2 = &s1[12..24];
           -- immutable borrow occurs here
 s1.push('*');
 ^^^^^^^^^^^^ mutable borrow occurs here
 println!("{s1} - {s2}");
                  -- immutable borrow later used here
```



&String / &str

```
fn foo (s:&String) {
    println!("{s}");
}

fn main() {
   let s1 = "teste".to_string();
   foo(&s1);
   // foo("outro teste"); Erro
   println!("{s1}");
}
```

```
fn foo (s:&str) {
   println!("{s}");
}


fn main() {
   let s1 = "teste".to_string();
   foo(&s1);
   foo("outro teste");
   println!("{s1}");
}
```